

CS & IT ENGINEERING



Computer Network - 1

Transport Layer

Lecture No. – 02

By - Abhishek Sir





Recap of Previous Lecture



Topic

Transport Layer Services





Topics to be Covered



Topic

UDP

Topic

TCP



Topic : UDP



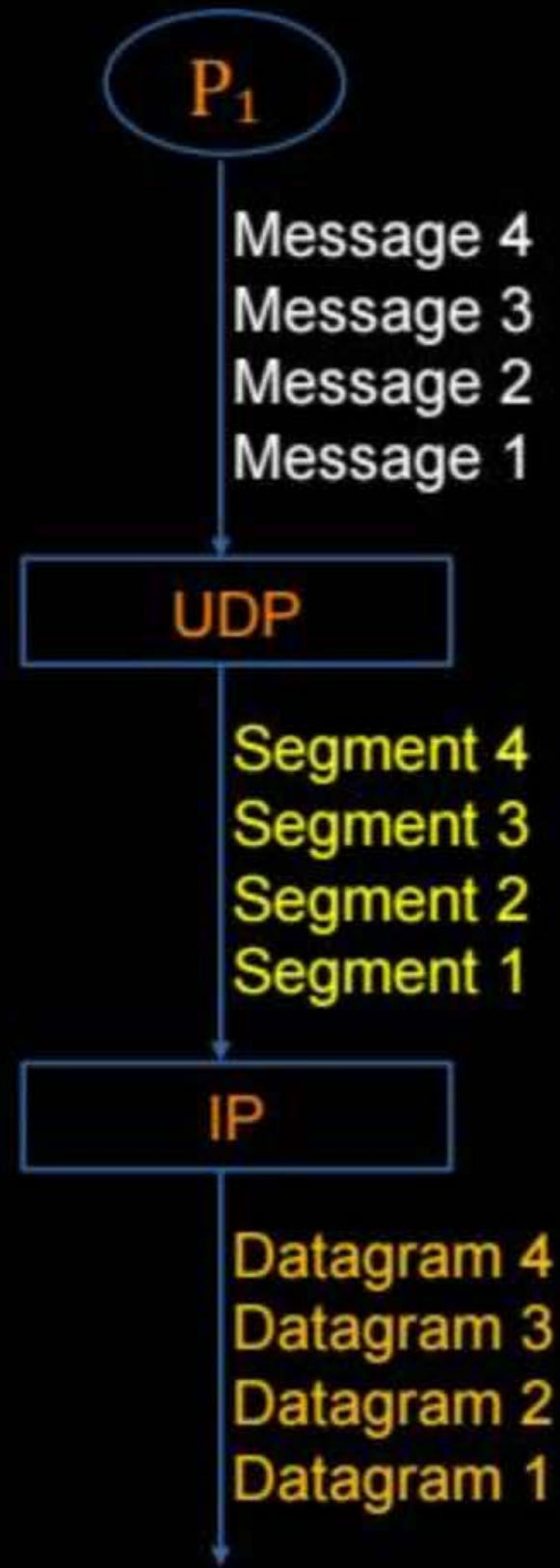
→ UDP : User Datagram Protocol ⇒ Basic Protocol

→ Provide 'Connection-less' and 'Unreliable' services
[Unordered delivery of messages, messages may be lost]

UDP ⇒ connection less and unreliable



Messages can be delivered in
any order





Topic : TCP

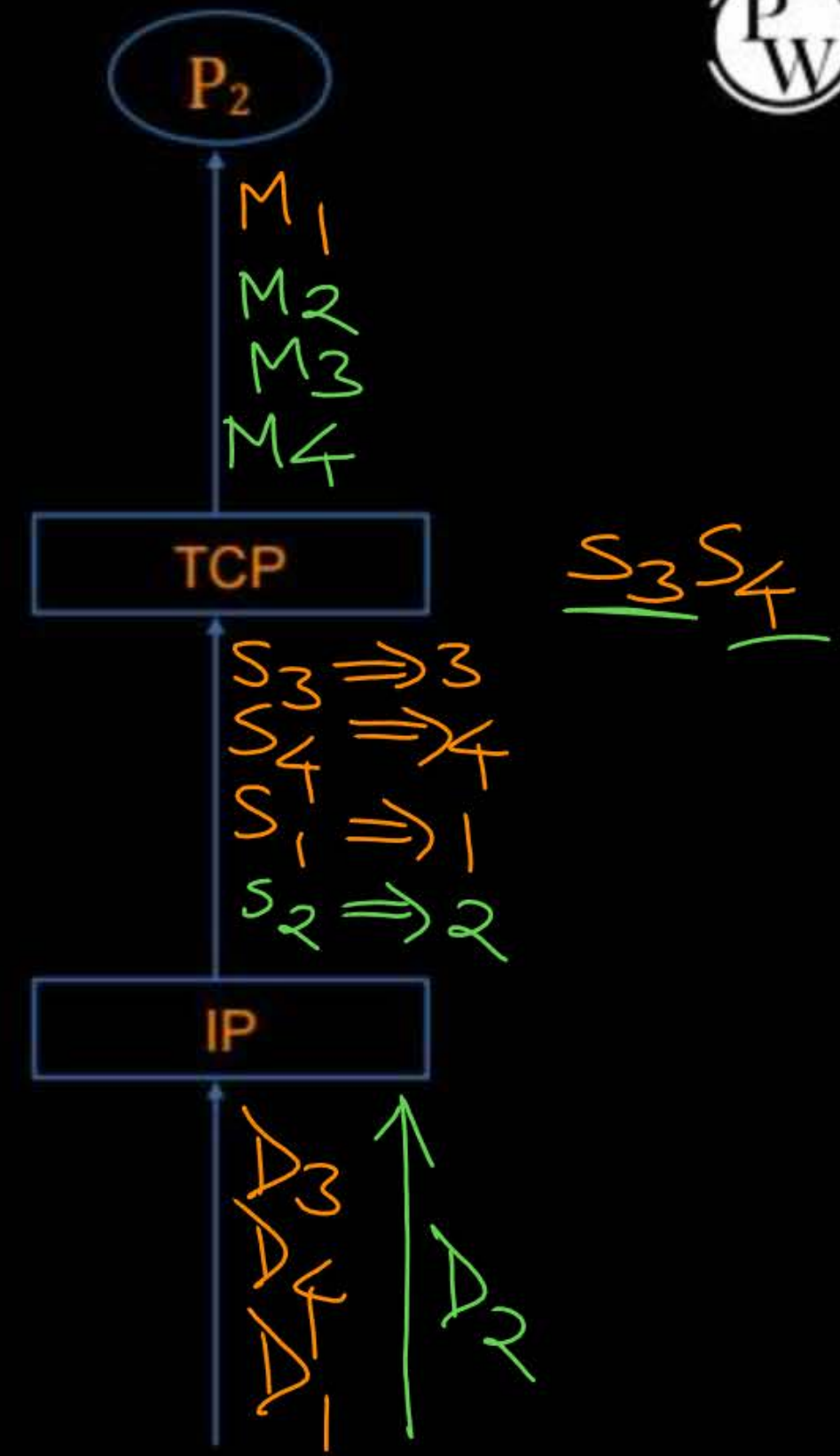
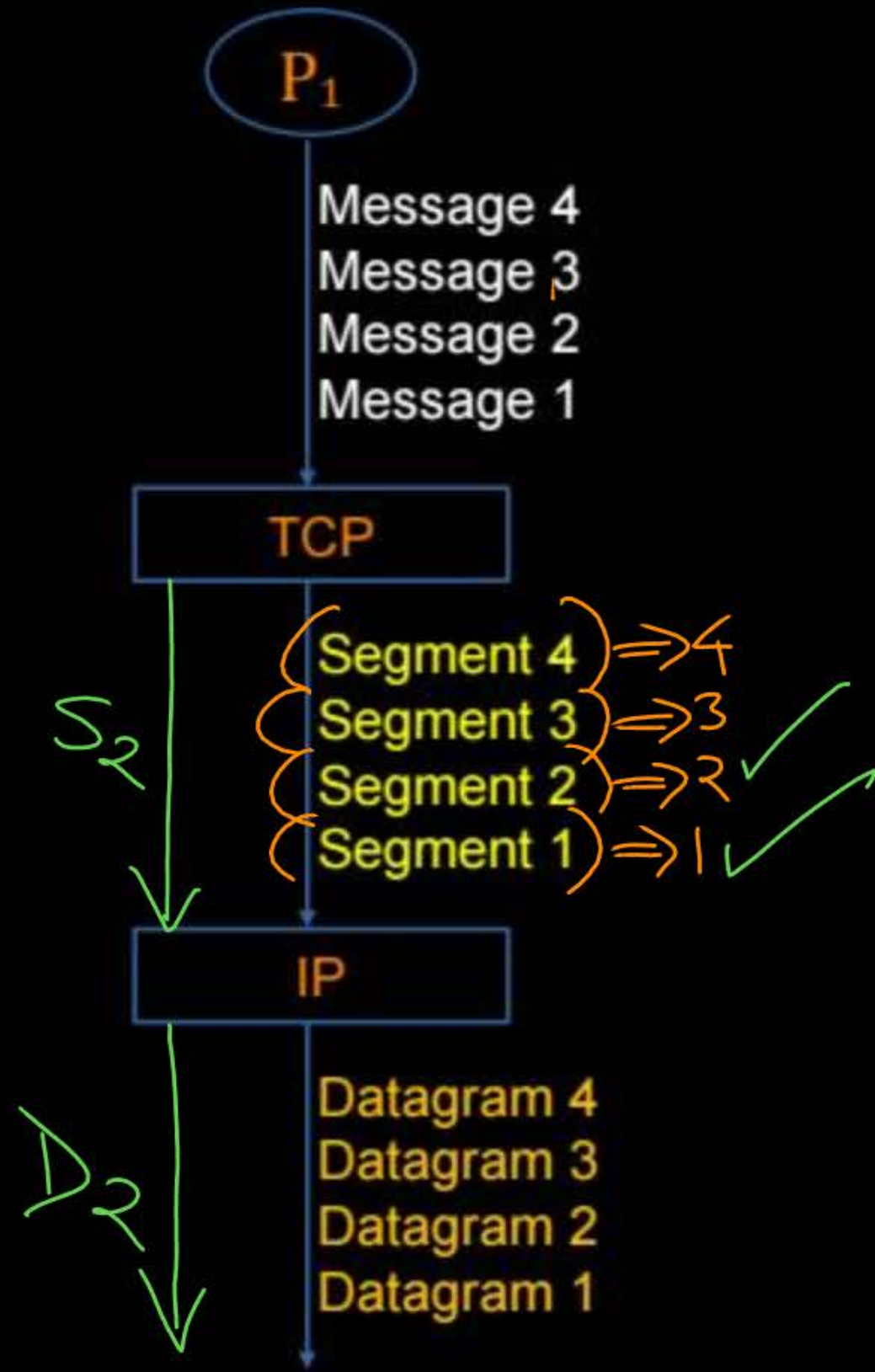


→ TCP : Transmission Control Protocol

→ Provide 'Connection-oriented' and 'Reliable' services, on top of connection-less and unreliable protocol.
[In-order delivery of messages]

[TCP = UDP + Extra Services]

TCP : Recover the loss, via Retransmission





Topic : UDP



=> Connection-less :

- > No handshaking between UDP sender and receiver
- > Each UDP segment handled independently of others
- > Delivery of messages can be any order to the communicating process

=> Unreliable :

- > Messages may be lost



Topic : UDP



=> Simple and Fast :

- > No connection state at sender and receiver
- > Small (8 bytes, fixed) header size
- > No any flow and congestion control



Topic : UDP



→ UDP : User Datagram Protocol

→ Prefer for shorter communication [like query and response]

→ UDP as transport protocol used by :

DNS, SNMP, HTTP/3, RIP and Real Time Multimedia [Streaming Multimedia]



Topic : Demultiplexing



Demultiplexing at receiver :

1. Connection-less demultiplexing ✓
2. Connection-oriented demultiplexing ⇒



Topic : Connection-Less Demultiplexing

→ UDP sockets identified by 2 tuples :

Destination IP address and Destination Port Number



Topic : UDP



UDP Client Process

P₁

SOCK_DGRAM
(UDP Socket)

UDP Server Process

P₂

SOCK_DGRAM
(UDP Socket)

IP(UDP(Message))

IP(UDP(Message))

UDP Client Process

P₃

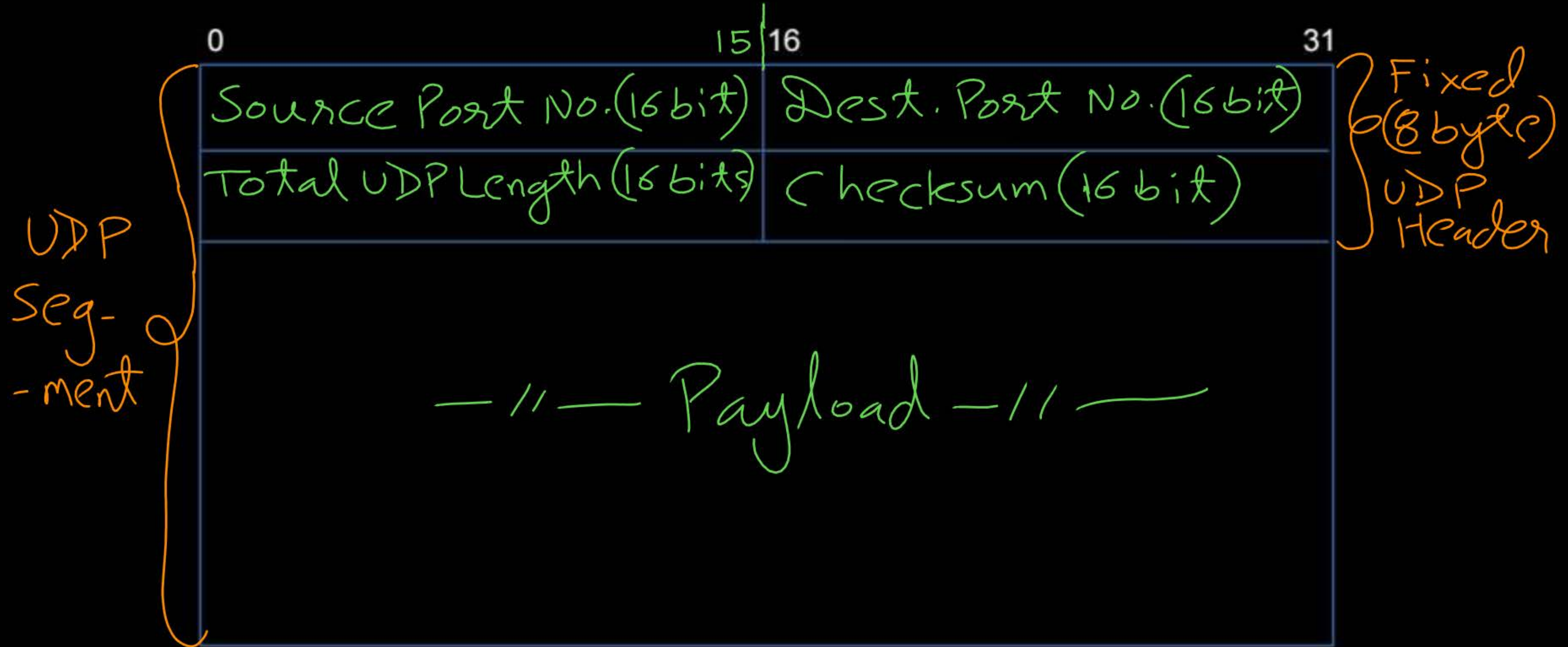
SOCK_DGRAM
(UDP Socket)

message

message



Topic : UDP Header





Topic : UDP Header



=> Source port number : 16-bits

=> Destination port number : 16-bits

=> Total Length : 16-bits

-> Size of UDP segment in bytes

-> Including 8 bytes header size

-> Maximum UDP segment size = $(2^{16} - 1)$ bytes

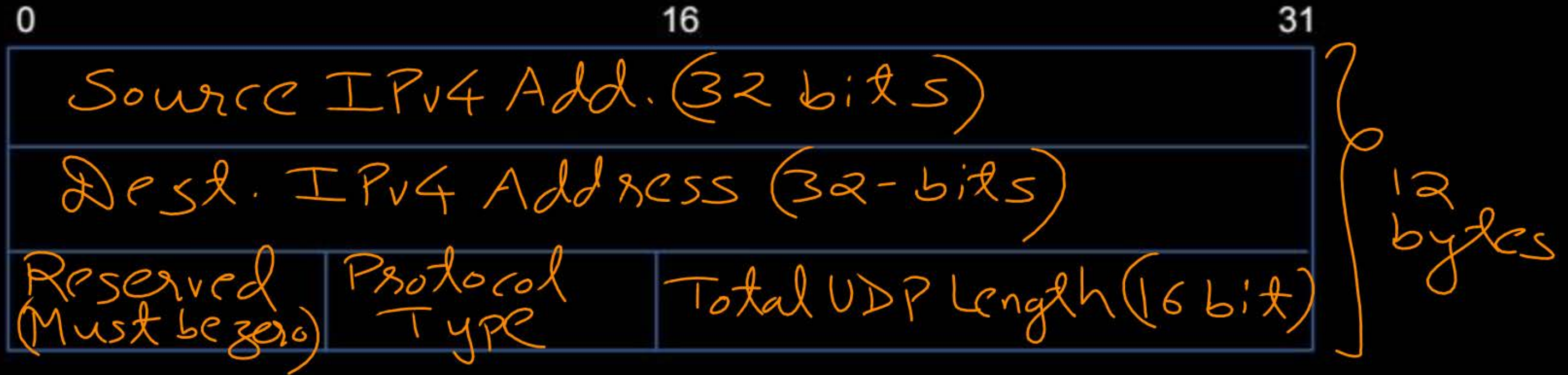
-> Maximum UDP payload size = $(2^{16} - 9)$ bytes

UDP Payload size = (Total length - 8) bytes



Topic : IPv4 Pseudo-Header

$\Rightarrow$ size = 12 bytes





Topic : IPv4 Pseudo-Header



=> Source IP Address : 32-bits

=> Destination IP Address : 32-bits

=> Reserved : 8-bits

→ Must be zero

=> Protocol : 8-bits

→ Same as IP protocol type

=> UDP Length : 16-bits

→ Size of UDP segment in bytes

→ 8 bytes header size plus payload size



Topic : UDP Header



0 16 31

IPv4
Pseudo
Header
-er
12 byte

Source IPv4 Address (32 bits)		
Destination IPv4 Address (32 bits)		
00 00 Reserved	Protocol = 17	[UDP Length (16 bits)]
Source Port Number (16 bits)		Destination Port Number (16 bits)
[Total Length (16 bits)]		[Checksum (16 bits)]
-//- Payload -//-		

UDP
Header
-er
8 byte

UDP
Segment



Topic : UDP Header



=> Checksum : 16-bits

-> 16-bit one's complement of the one's complement sum of all 16-bit words in UDP segment including pseudo header

-> Error detection is optional in UDP, it depends upon the application

-> If application wants to disable error detection process then sender set all the 16 bits are zero of checksum field



Topic : TCP



=> Connection-oriented :

↓
-> Need to establish (logical) connection between TCP sender and receiver

◆ 3-way handshaking for connection establishment

◆ 4-way handshaking for connection close

-> In order delivery of messages to the communicating process



Topic : TCP



=> Reliable :

-> Recover lost messages, via retransmission

=> Complex and Slow :

-> Need to maintain connection state at TCP sender and receiver

-> Provide flow and congestion control



Topic : TCP



→ TCP : Transmission Control Protocol

→ Full-duplex and point-to-point logical connection

→ Provide Flow, Error and Congestion Control

→ Prefer for long communication



2 mins Summary



Topic

UDP



Topic

TCP





THANK - YOU

